

MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Department of Computer Science

CSC2201 — Software Engineering

SOFTWARE ENGINEERING REPORT

School Tracking System (STS)

GROUP 13 — COMPUTER SCIENCE

Registration No.	Name
2024/BCS/047/PS	ASASIRA IAN TUSIIME
2024/BCS/004	KEMIGISHA SANDRA
2024/BCS/109/PS	AMON MUHWEZI
2024/BCS/178/PS	AINEMBABAZI DAPHINE
2024/BCS/061/PS	BEEKA ARNOLD
2024/BCS/020/PS	AHUMUZA ROVIN

GitHub Repository:

<https://github.com/arnobeek/SchoolTrackingSystem>



1. Introduction

1.1 Project Background

Secondary schools in Uganda and across sub-Saharan Africa continue to operate on predominantly manual administrative systems. Attendance registers, examination records, and fee ledgers are maintained on paper, creating significant inefficiencies that erode the quality of institutional management. The School Tracking System (STS) is a proposed digital platform that directly addresses these systemic challenges.

At the core of the problem is the inability to verify whether teachers are physically present in school during teaching hours. Without reliable verification, there is no guarantee that classes are conducted as scheduled, leaving students without adequate instructional time. Administrators rely on self-reported attendance data that cannot be independently validated, and parents have no visibility into what is happening inside the school.

This report presents the complete software engineering analysis, modelling, and design of the School Tracking System — a solution that combines GPS geofencing, fingerprint biometric authentication, and cloud-based reporting to digitise and secure the core administrative functions of a secondary school.

1.2 Problem Statement

The following critical problems motivated the development of this system:

- Manual attendance recording by teachers is unreliable and easily falsified, with no mechanism to confirm that a teacher was physically on school grounds.
- Student attendance tracking is inconsistent across classes and streams, making it impossible to identify chronic absenteeism patterns early.
- Examination result management is slow and error-prone when conducted manually, delaying feedback to students and parents.
- Fee management lacks real-time visibility — accountants maintain ledgers that are not instantly accessible to administrators or parents.
- Communication between the school and parents is poor; notifications about attendance, results, and fees are delayed or never delivered.
- There is no accountability mechanism for teachers, leaving head teachers unable to act on absence patterns without manual record investigation.

1.3 Project Objectives

The School Tracking System aims to achieve the following specific objectives:

- Implement a secure biometric (fingerprint) and GPS-based teacher attendance system that verifies both identity and physical location.
- Digitise student attendance tracking so that class-by-class records are automatically stored and accessible.
- Provide a structured examination and results management module with grade computation and performance analytics.
- Develop a real-time fee management module with automated balance calculation and payment history.

- Create automated SMS and email notification pipelines to keep parents informed of attendance, results, and fees.
- Deliver role-based dashboards for administrators, teachers, parents, and students with appropriate data access controls.

1.4 Scope of the System

The system is scoped to serve a single secondary school as the primary deployment unit, with architecture designed to support multi-school deployment in future increments. The scope includes:

- Teacher and student attendance management with GPS geofencing and fingerprint verification.
- Examination scheduling, result entry, and academic performance reporting.
- Fee recording, payment tracking, and outstanding balance reporting.
- Automated notifications to parents via SMS and email.
- Administrative dashboards with exportable reports.

Out of scope for the current version: integration with the national NSIN (National Student Identification Number) database, payroll management, and library management.

2. Stakeholder Analysis

A comprehensive stakeholder analysis ensures that the system is built to meet the real needs of all parties who will interact with or be affected by the system.

2.1 Primary Stakeholders

Primary stakeholders are those who directly interact with the system on a regular basis:

Stakeholder	Role	Needs / Interests
Teacher	Attendance logging	Easy fingerprint + GPS login; view own attendance history; mark student attendance
Student	Academic participant	View personal attendance records, examination results, and fee balances
Parent/Guardian	Information consumer	Receive real-time SMS/email notifications on child's attendance, results, and fees
Administrator	System manager	Full access to reports, user management, and configuration of school settings
Head Teacher	School executive	Executive dashboards; teacher accountability reports; academic performance summaries
Accountant	Fee manager	Record payments, generate fee statements, track arrears, and export financial reports

2.2 Secondary Stakeholders

Secondary stakeholders do not directly use the system but are significantly impacted by its outputs:

- Ministry of Education — benefits from aggregated school performance data that can inform national education policy and school inspection processes.
- System Developers — responsible for the design, development, testing, and deployment of the system; require clear and complete requirements documentation.
- IT Support Staff — responsible for day-to-day system maintenance, user account management, hardware upkeep (fingerprint scanners, GPS modules), and technical troubleshooting.
- External Auditors — may require access to attendance and fee records for institutional accountability reviews.

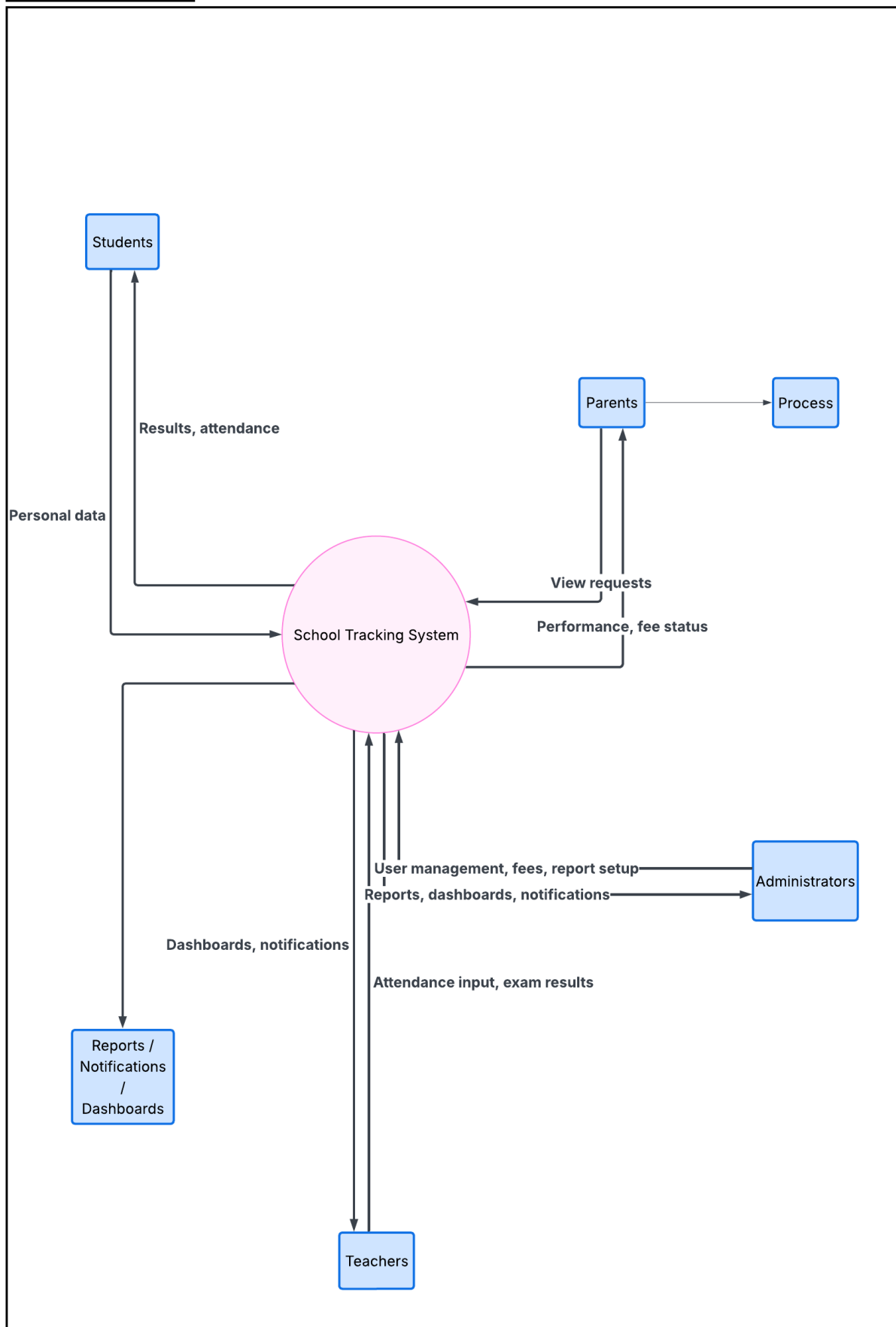
3. Process Models

Process models provide graphical and structured representations of how the system processes information. The following diagrams have been developed for the School Tracking System: a Context Diagram, Level 1 DFD, Level 2 DFD, and an Activity Diagram.

3.1 Context Diagram

The Context Diagram (also called a Level 0 DFD) treats the entire School Tracking System as a single process — a black box — and illustrates how it interacts with external entities. This diagram establishes the system boundary and identifies all entities that send data to or receive data from the system.

School Tracking System Context Diagram



External Entities and Their Interactions

External Entity	Data Sent to System	Data Received from System
Teacher	Fingerprint scan, GPS coordinates, student marks	Authentication confirmation, attendance status, schedule
Student	Login credentials, exam answers (where applicable)	Attendance records, exam results, fee balance
Parent/Guardian	Contact details (registration)	SMS/email notifications: attendance, results, fees
Administrator	Configuration settings, user data, school parameters	Full system reports, analytics dashboards, audit logs
Accountant	Payment receipts, fee structures	Fee statements, payment history, outstanding balances

The Context Diagram shows that the School Tracking System serves as the central hub through which all school administrative data flows. No external entity communicates directly with another; all data exchanges are mediated by the system, ensuring data integrity, access control, and audit capability.

[Figure 3.1: Context Diagram (Level 0 DFD)]
School Tracking System as central hub with all external entities

Figure 3.1: Context Diagram — School Tracking System

3.2 Level 1 Data Flow Diagram (DFD)

The Level 1 DFD decomposes the system into its five major functional processes. Each process is shown with its data inputs, outputs, associated data stores, and the external entities that trigger or consume the data.

#	Process	Inputs	Outputs	Data Store
1	Manage Attendance	Fingerprint, GPS data	Attendance status, reports	Attendance DB
2	Manage Exams & Results	Exam marks from teachers	Grade reports, transcripts	Results DB
3	Manage Fees	Payment data from accountant	Fee statements, balances	Fees DB
4	Generate Reports	Data from all DBs	Dashboards, exports	Reports DB
5	Send Notifications	Triggers from processes 1–4	SMS/email alerts	Notification Log

Process 1 (Manage Attendance) is the most complex, as it integrates biometric validation, GPS geofencing logic, and time-based rules. Process 4 (Generate Reports) is a cross-cutting concern that aggregates data from all data stores. Process 5 (Send Notifications) is event-driven, triggered by threshold conditions in other processes — for example, when student attendance falls below 75%, or when a fee payment is overdue.

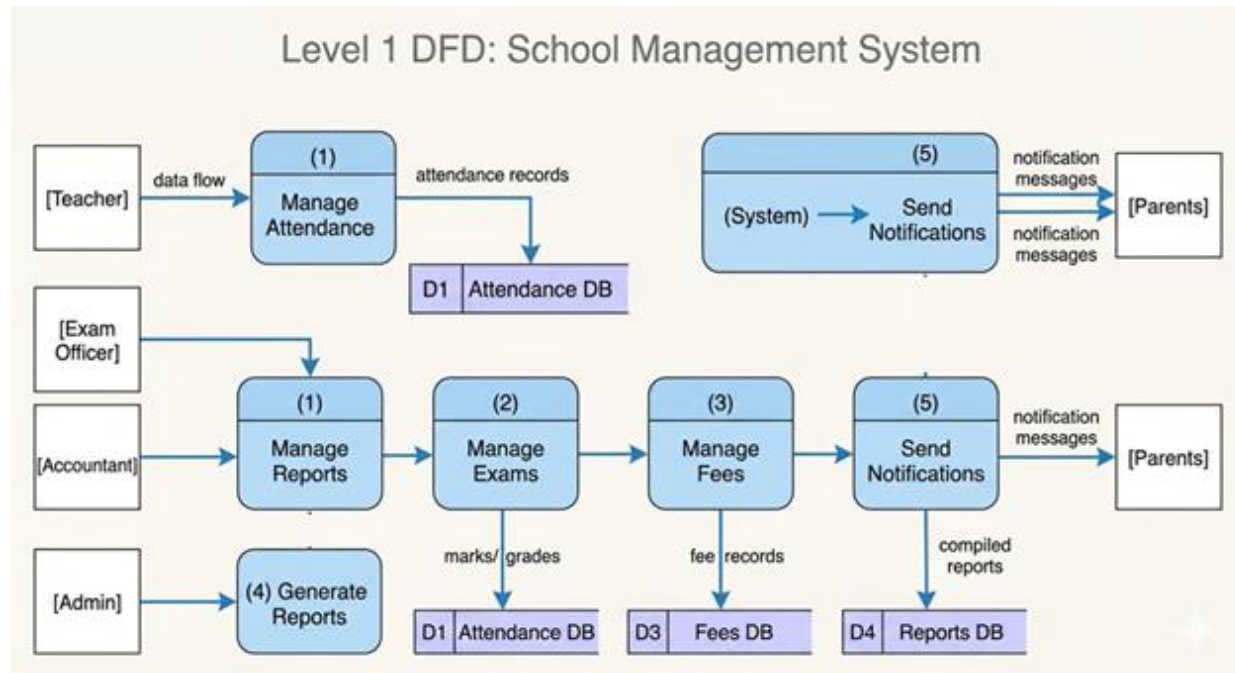


Figure 3.2: Level 1 Data Flow Diagram — School Tracking System

3.3 Level 2 Data Flow Diagram — Attendance Management

The Level 2 DFD provides a detailed breakdown of Process 1 (Manage Attendance) from the Level 1 DFD. This level reveals the internal logic of the teacher attendance verification pipeline, which is the most novel aspect of the School Tracking System.

Sub-processes within Attendance Management

- **Capture Fingerprint** — The teacher places their finger on the registered biometric scanner. The device captures the fingerprint image and converts it to a numerical template.
- **Validate Fingerprint Identity** — The captured fingerprint template is compared against the stored template in the Biometric/User Database. If the match score exceeds the acceptance threshold, identity is confirmed. If not, the attempt is rejected and logged.
- **Acquire GPS Coordinates** — The system (via the teacher's device or a fixed school terminal) captures the current GPS latitude and longitude coordinates.
- **Apply Geofence Validation** — The GPS coordinates are compared against the school's defined geofence polygon (a GPS boundary around the school premises). If the teacher is within the boundary, location is validated. If outside, access is denied and a security event is logged.
- **Determine Attendance Status** — If both fingerprint and GPS validation pass, the system timestamps the record and determines the status: Present, Late (if after the grace period), or Absent (if the window has closed).

- **Record Attendance** — The confirmed attendance event is written to the Attendance Database with teacher ID, timestamp, GPS coordinates, and status.
- **Trigger Notification (if applicable)** — If a teacher is absent after the expected time, the system triggers an alert to the Head Teacher dashboard and optionally sends an SMS.

Key Design Decision

The GPS geofence is defined as a polygon (not a simple radius circle) to accommodate irregularly shaped school compounds. The geofence coordinates are configured by the Administrator during system setup and can be updated without system redeployment. The default attendance window opens 30 minutes before the first lesson and closes 15 minutes after the first lesson begins.

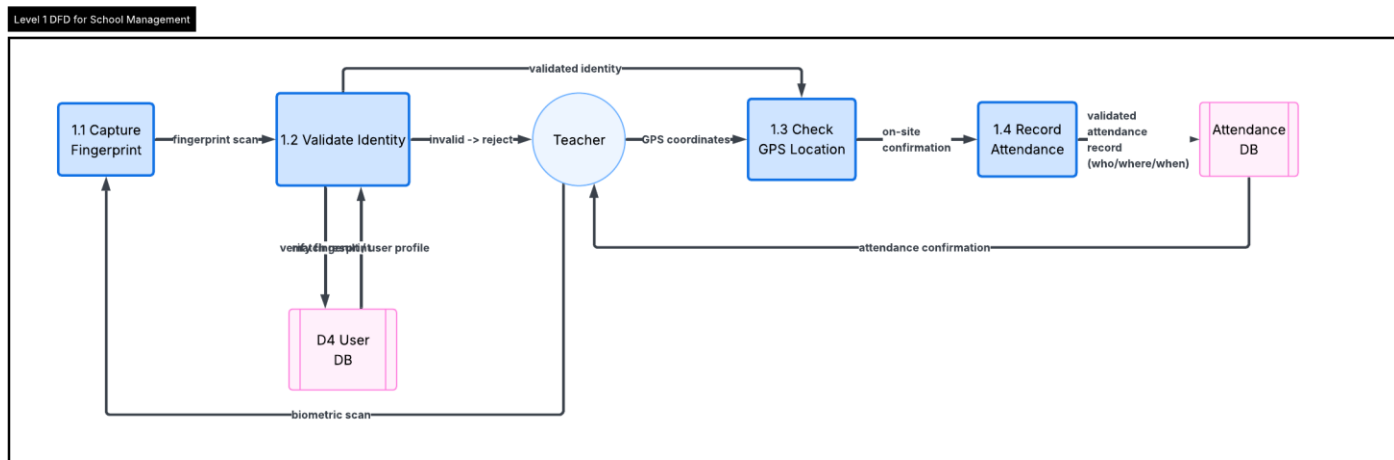


Figure 3.3: Level 2 DFD — Attendance Management Process

3.4 Activity Diagram — Teacher Attendance Workflow

The Activity Diagram illustrates the complete workflow of the teacher attendance process as a sequence of decisions and actions, from system login through to report generation. Activity diagrams are particularly useful for depicting the conditional logic and branching that governs complex workflows.

Workflow Narrative

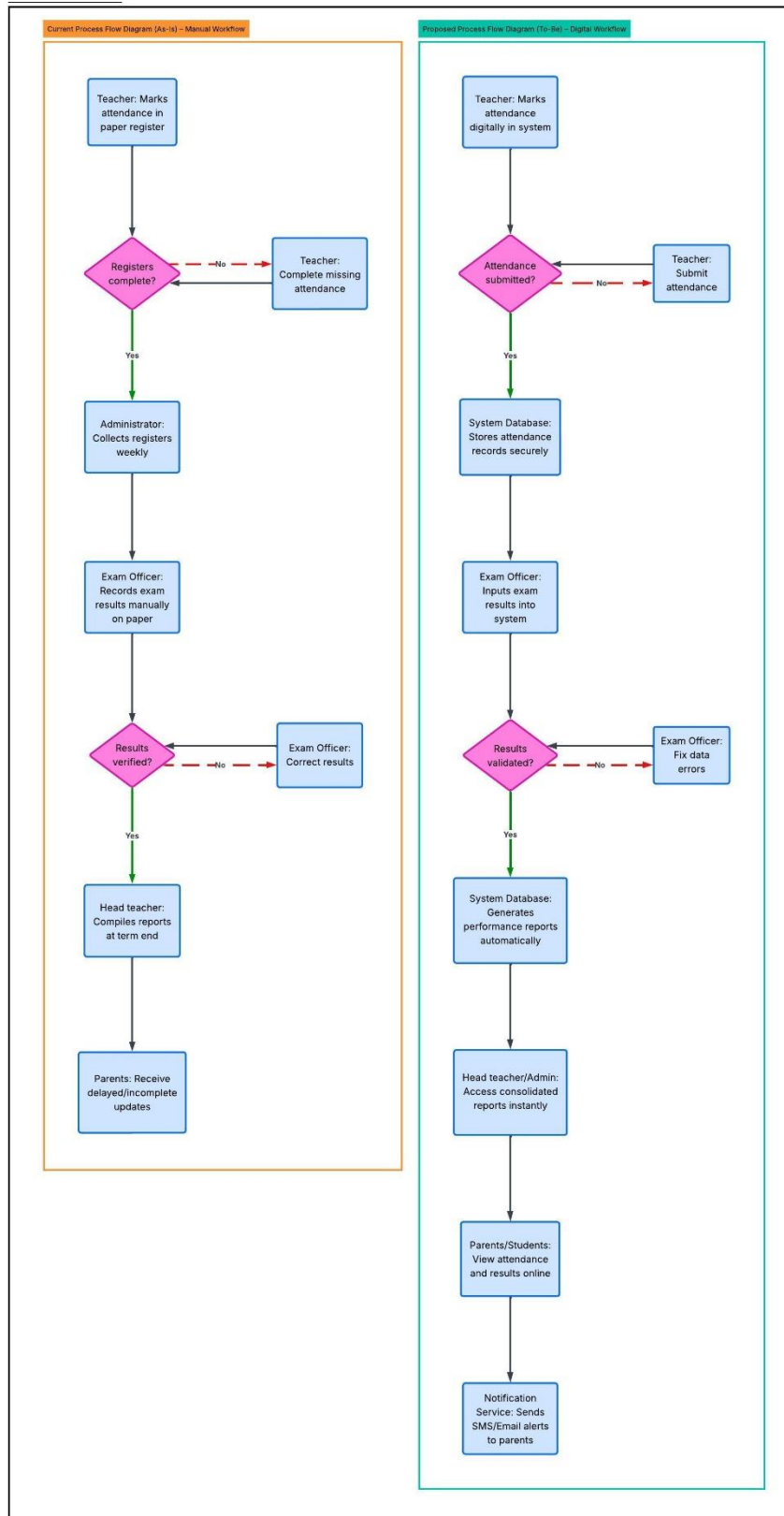
The teacher begins by logging into the system using their username and password. Upon successful authentication, the system prompts for biometric verification. The teacher presents their registered finger to the fingerprint scanner, and the system performs a live comparison against the stored template.

If the fingerprint does not match, the system records a failed attempt and allows up to three retries before locking the account for 30 minutes. This prevents brute-force or proxy attempts. If the fingerprint is validated, the system immediately queries the device's GPS module.

A geofence check is performed in real time. If the teacher's GPS coordinates fall outside the defined school boundary, the system rejects the attendance event, records a location violation, and notifies the administrator. The teacher is not marked present regardless of fingerprint success.

If both checks pass, the system marks the teacher as Present (or Late, depending on time) and saves the record to the Attendance Database. A confirmation message is displayed to the teacher. The attendance data becomes immediately available in the administrator's dashboard and contributes to the automated daily attendance report.

Step	Description
1. Login	Teacher enters username and password; system authenticates against User DB
2. Fingerprint Capture	Teacher presents finger to biometric scanner; system captures live template
3. Identity Validation	Live template compared to stored template; match threshold applied
4. GPS Acquisition	Device acquires GPS coordinates from satellite or network location
5. Geofence Check	Coordinates validated against school boundary polygon
6. Status Assignment	System assigns Present/Late/Rejected based on time and validation results
7. DB Write	Attendance record persisted with timestamp, GPS, and teacher ID
8. Report Update	Dashboard and notification system updated in real time



4. Data Models

Data models define the structure of data within the system, the relationships between data entities, and the constraints governing data integrity. The School Tracking System employs a relational data model with clear entity definitions, attribute specifications, and cardinality relationships.

4.1 Entity Identification and Description

The following entities have been identified from the system requirements and stakeholder analysis:

4.1.1 Student

The Student entity represents a registered learner within the school. Each student has a unique identifier and is associated with a class and stream. Student records are the central reference point for attendance, results, and fee data.

Attribute	Data Type	Description
student_id (PK)	INT AUTO_INCREMENT	Unique identifier for each student
full_name	VARCHAR(100)	Student's full legal name
date_of_birth	DATE	Student's date of birth for age verification
class_id (FK)	INT	References the Class entity (e.g., S3 West)
nsin	VARCHAR(20)	National Student Identification Number (optional)
parent_contact	VARCHAR(15)	Primary parent/guardian phone number for SMS
parent_email	VARCHAR(100)	Parent email address for email notifications
enrollment_date	DATE	Date the student enrolled at the school
is_active	BOOLEAN	Flags whether student is currently enrolled

4.1.2 Teacher

The Teacher entity stores identity and biometric data for all members of teaching staff. The fingerprint_template attribute is the core of the biometric authentication mechanism.

Attribute	Data Type	Description
teacher_id (PK)	INT AUTO_INCREMENT	Unique identifier for each teacher
full_name	VARCHAR(100)	Teacher's full name
national_id	VARCHAR(20)	National identification number
fingerprint_template	BLOB	Encrypted biometric fingerprint template
subject_id (FK)	INT	Primary subject the teacher is assigned to

Attribute	Data Type	Description
phone_number	VARCHAR(15)	Contact number for alerts
hire_date	DATE	Date of employment at the school
is_active	BOOLEAN	Whether teacher is currently employed

4.1.3 Attendance

The Attendance entity captures every attendance event — for both teachers and students. The inclusion of GPS coordinates and fingerprint_verified flag creates an auditable, tamper-resistant record.

Attribute	Data Type	Description
attendance_id (PK)	INT AUTO_INCREMENT	Unique record identifier
user_type	ENUM('teacher','student')	Distinguishes teacher from student attendance
user_id (FK)	INT	References teacher_id or student_id
date	DATE	Calendar date of attendance record
check_in_time	TIMESTAMP	Exact time the attendance was recorded
status	ENUM('present','late','absent')	Final attendance status
gps_latitude	DECIMAL(10,8)	GPS latitude at time of check-in
gps_longitude	DECIMAL(11,8)	GPS longitude at time of check-in
fingerprint_verified	BOOLEAN	Whether fingerprint validation passed
geofence_passed	BOOLEAN	Whether GPS geofence check passed

4.1.4 Examination and Result

The Exam entity defines an examination event, while the Result entity stores the marks and computed grades for each student per exam. The grading logic (computing letter grades and remarks) is handled at the application layer based on configurable grade boundaries.

Attribute (Exam)	Data Type	Description
exam_id (PK)	INT	Unique exam identifier
exam_name	VARCHAR(100)	e.g., 'End of Term 1 Examinations 2025'
term	ENUM('1','2','3')	Academic term
year	YEAR	Academic year
class_id (FK)	INT	Class the exam applies to

Attribute (Result)	Data Type	Description
result_id (PK)	INT	Unique result record identifier
student_id (FK)	INT	References the Student entity
exam_id (FK)	INT	References the Exam entity
subject_id (FK)	INT	The subject this result relates to
marks_obtained	DECIMAL(5,2)	Raw marks scored by the student
grade	CHAR(2)	Computed letter grade (A, B, C, D, F)
remarks	VARCHAR(50)	e.g., Distinction, Credit, Pass, Fail

4.1.5 Fees

The Fee entity records individual payment transactions. Each fee record is linked to a student and a fee structure, making it possible to compute outstanding balances and generate statements automatically.

Attribute	Data Type	Description
payment_id (PK)	INT	Unique payment record identifier
student_id (FK)	INT	Student this payment relates to
fee_structure_id (FK)	INT	References the configured fee structure
amount_paid	DECIMAL(10,2)	Amount paid in this transaction (UGX)
payment_date	DATE	Date payment was received
receipt_number	VARCHAR(30)	Official receipt number from accountant
term	ENUM('1','2','3')	Term this payment applies to
outstanding_balance	DECIMAL(10,2)	Remaining amount owed after this payment
recorded_by (FK)	INT	Accountant who recorded the payment

4.2 Entity Relationship Diagram — Relationships

The ERD defines the following key relationships between entities:

- Student (1) — records → (M) Attendance: One student has many attendance records across different dates and periods.
- Teacher (1) — logs → (M) Attendance: One teacher generates many attendance records over their tenure.
- Student (1) — achieves → (M) Result: One student can have results across many subjects and exams.
- Exam (1) — contains → (M) Result: One exam has results for many students across many subjects.

- Student (1) — makes → (M) Fee Payment: One student can have many fee payment records across terms.
- User (1) — has role → (1) UserRole: Each system user has exactly one role (Admin, Teacher, Accountant, Parent, Student).
- Administrator — manages → School Configuration: Geofence coordinates, grade boundaries, fee structures, and notification rules.

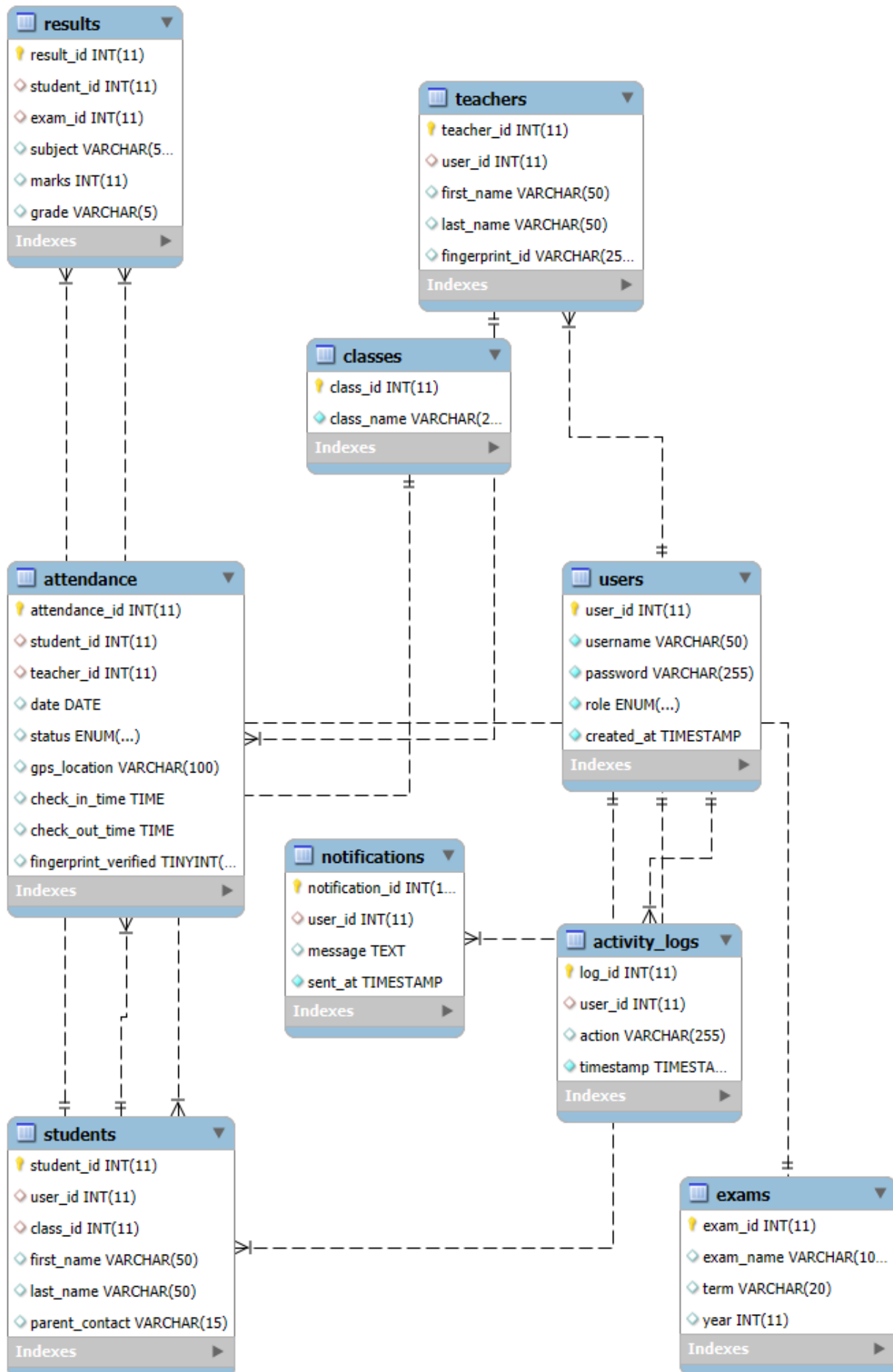


Figure 4.1: Entity Relationship Diagram — School Tracking System

4.3 Class Diagram Overview

The Class Diagram for the School Tracking System maps the data model to object-oriented classes used in the application layer. Key classes include: User (abstract parent), Teacher (extends User), Student (extends User), AttendanceService, ExamService, FeeService, NotificationService, GeofenceValidator, and BiometricValidator. The service classes encapsulate business logic and interact with the database through Repository classes, following the Repository Pattern for

5. User Interface and Interaction Models

User interface models define how actors interact with the system. The following models have been developed: a Use Case Diagram, Sequence Diagrams for key interactions, Wireframes for key screens, and a Screen Navigation Diagram.

5.1 Use Case Diagram

The Use Case Diagram identifies all system actors and the use cases (functions) available to each. It defines the boundaries of the system from the user's perspective.

Actors and Their Use Cases

Actor	Use Cases
Teacher	Log In, Verify Fingerprint, Mark Own Attendance, Enter Student Marks, View Own Attendance History, View Timetable
Student	Log In, View Own Attendance, View Exam Results, View Fee Balance, View Timetable
Parent	Receive Notifications, View Child's Attendance, View Child's Results, View Child's Fee Balance (via parent portal)
Admin	Manage Users, Configure Geofence, View All Reports, Manage Classes, Configure Fee Structure, Override Attendance, Generate Export
Accountant	Record Fee Payment, View Fee Ledger, Generate Fee Statement, View Outstanding Balances
Head Teacher	View Teacher Attendance Summary, View School Performance Report, View Absence Alerts, Approve System Configurations

Include and Extend Relationships

- Mark Own Attendance <<include>> Verify Fingerprint — Fingerprint verification is a mandatory sub-step of the attendance marking process.

- Mark Own Attendance <<include>> GPS Geofence Check — GPS validation is also mandatory and always runs as part of attendance marking.
- Log In <<extend>> Reset Password — Password reset is an optional extension that is triggered only when the user cannot authenticate.
- Generate Report <<extend>> Export to PDF/Excel — Exporting is an optional extension of the report viewing use case.

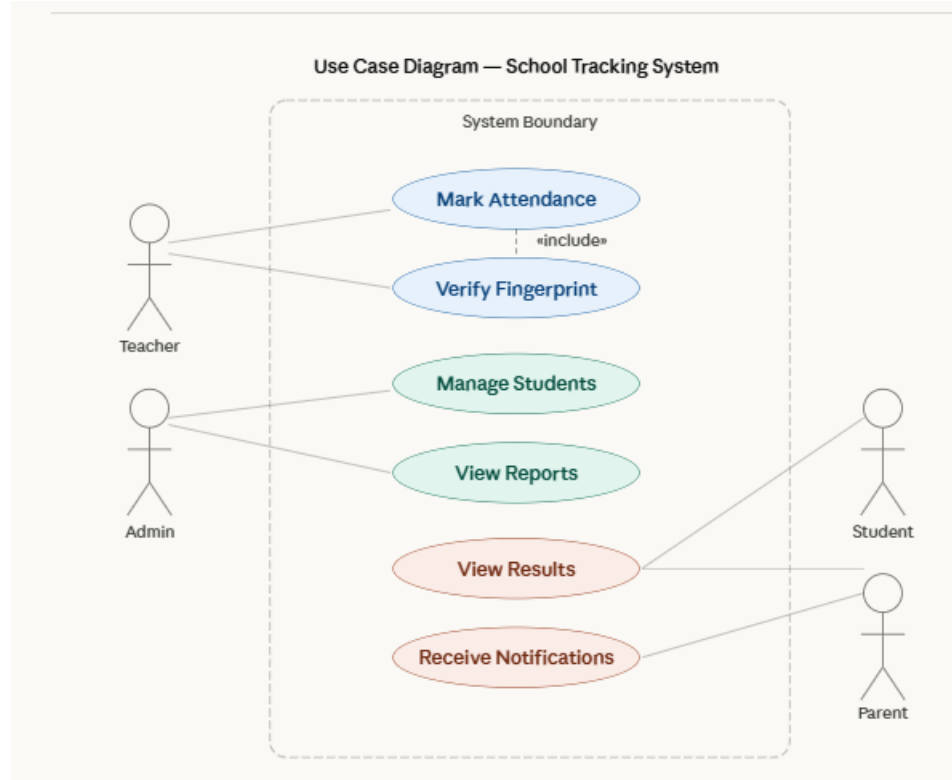


Figure 5.1: Use Case Diagram — School Tracking System

5.2 Sequence Diagrams

Sequence diagrams illustrate the chronological order of messages exchanged between actors and system components. Two key scenarios are documented here.

5.2.1 Teacher Attendance Marking Sequence

The following sequence describes the interaction between the Teacher, the Mobile/Terminal Device, the Authentication Service, the Geofence Service, and the Database when a teacher marks attendance:

- Teacher → System: POST /login (username, password)
- System → User DB: Validate credentials
- User DB → System: Credentials confirmed
- System → Teacher: Return authentication token (JWT)
- Teacher → Biometric Scanner: Present fingerprint
- Biometric Scanner → BiometricService: Send fingerprint template
- BiometricService → User DB: Fetch stored template
- BiometricService: Compare templates (match score $\geq$ threshold?)

- If match fails: BiometricService → Teacher: Reject, log attempt
- If match passes: Device → GeofenceService: Send GPS coordinates
- GeofenceService: Check coordinates against school boundary polygon
- If outside boundary: GeofenceService → System: Location rejected; log event
- If inside boundary: GeofenceService → AttendanceService: Confirm location
- AttendanceService → DB: Write attendance record (status, time, GPS, verified flags)
- AttendanceService → NotificationService: Trigger if anomaly detected
- System → Teacher: Display confirmation screen

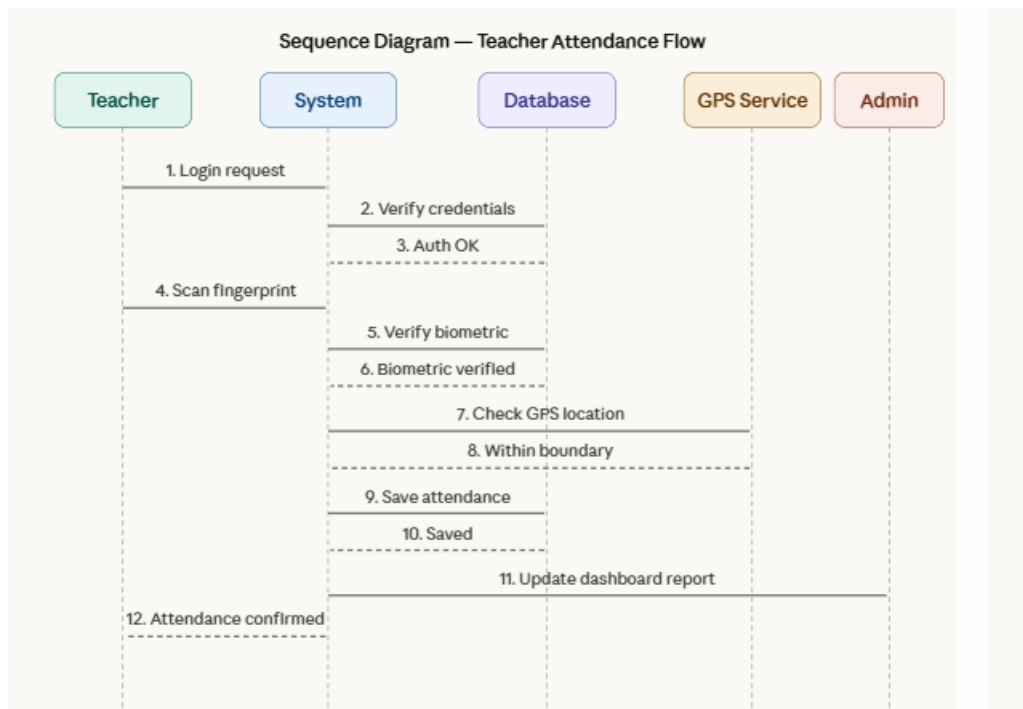


Figure 5.2: Sequence Diagram — Teacher Attendance Marking

5.2.2 Parent Notification Sequence

This sequence describes how a parent receives an automated SMS when their child is marked absent:

- AttendanceService: Student marked absent after attendance window closes
- AttendanceService → NotificationService: TRIGGER_ABSENCE_ALERT(student_id, date)
- NotificationService → Student DB: Fetch parent_contact for student_id
- NotificationService → SMS Gateway (e.g., Africa's Talking): Send SMS message
- SMS Gateway → Parent Phone: Deliver SMS: 'Dear Parent, [Name] was absent on [Date].'
- NotificationService → Notification Log DB: Record notification event with timestamp
- Admin Dashboard: Updated with notification delivery status in real time

5.3 User Interface — Screenshots

The following figures present the actual implemented screens of the School Tracking System. These screenshots illustrate how the system design described in this report has been materialised into a functional application.

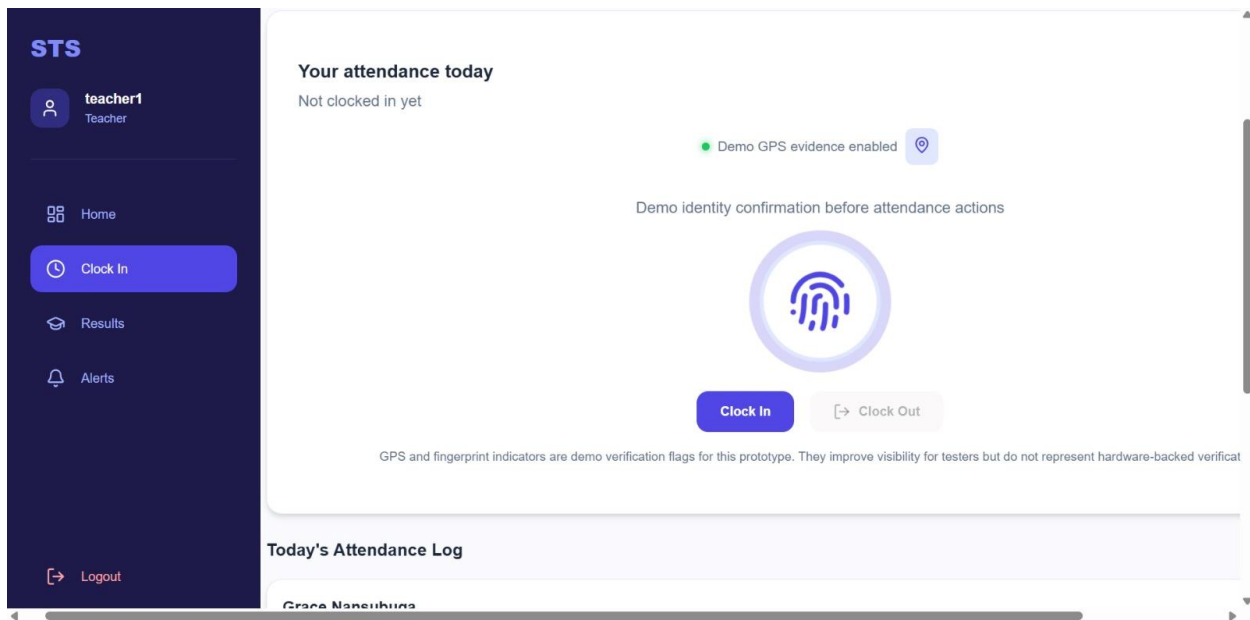


Figure 5.3: Teacher Clock-In Interface — Biometric and GPS Attendance Marking

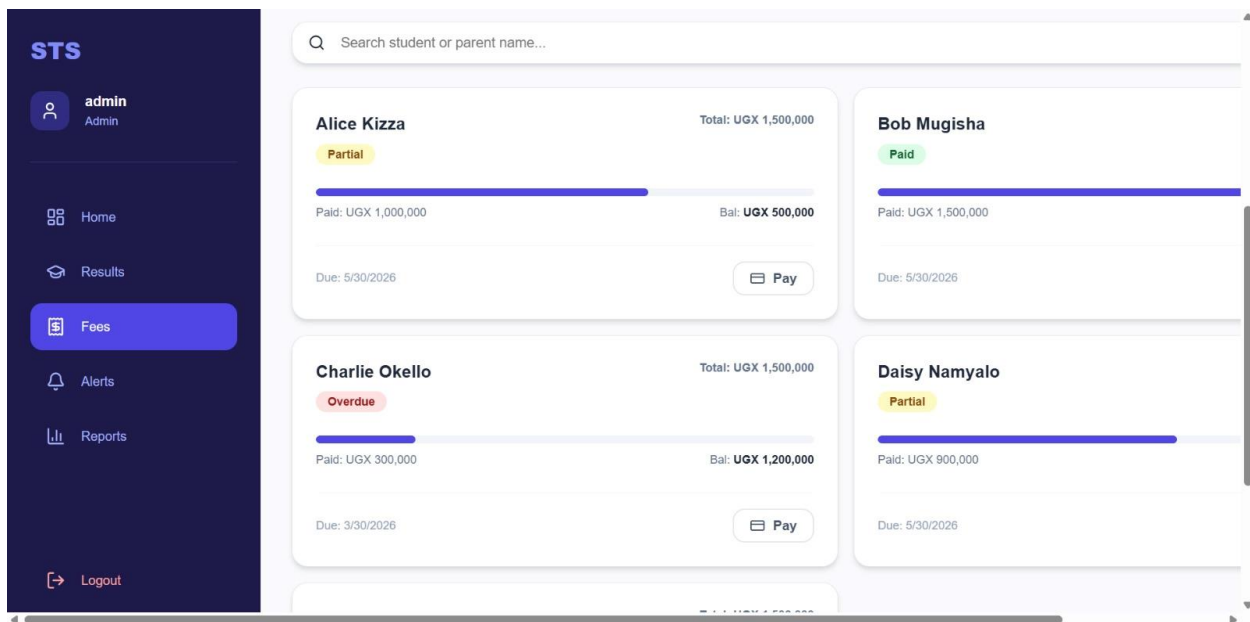


Figure 5.4: Administrator Fee Management Screen

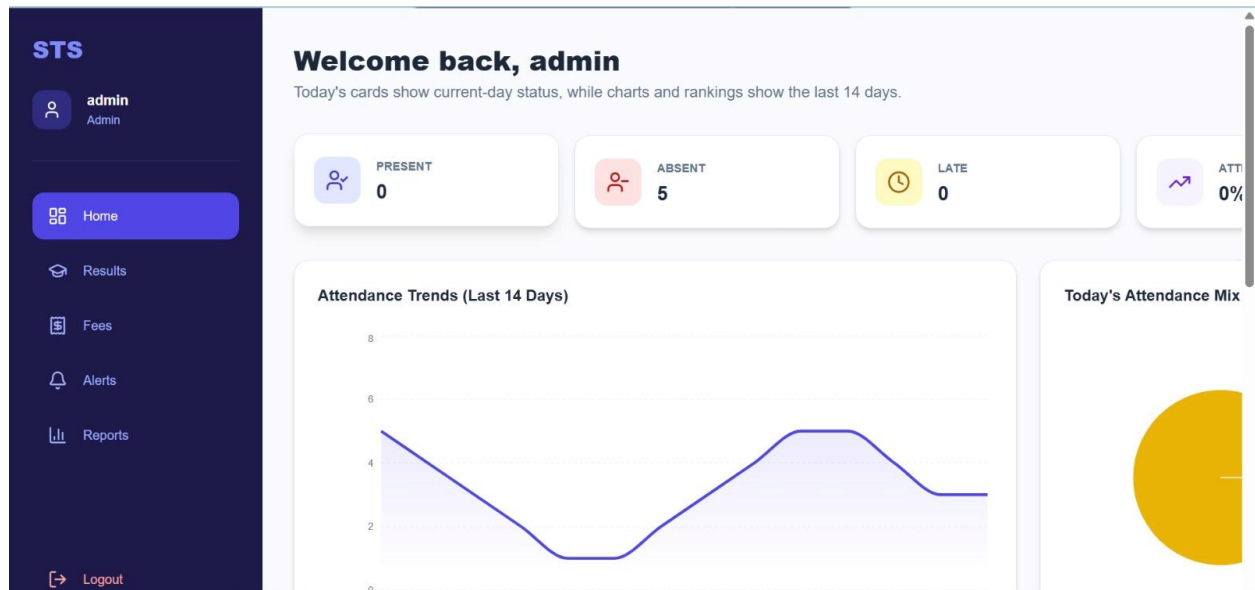


Figure 5.5: Administrator Dashboard — Attendance Trends and Analytics

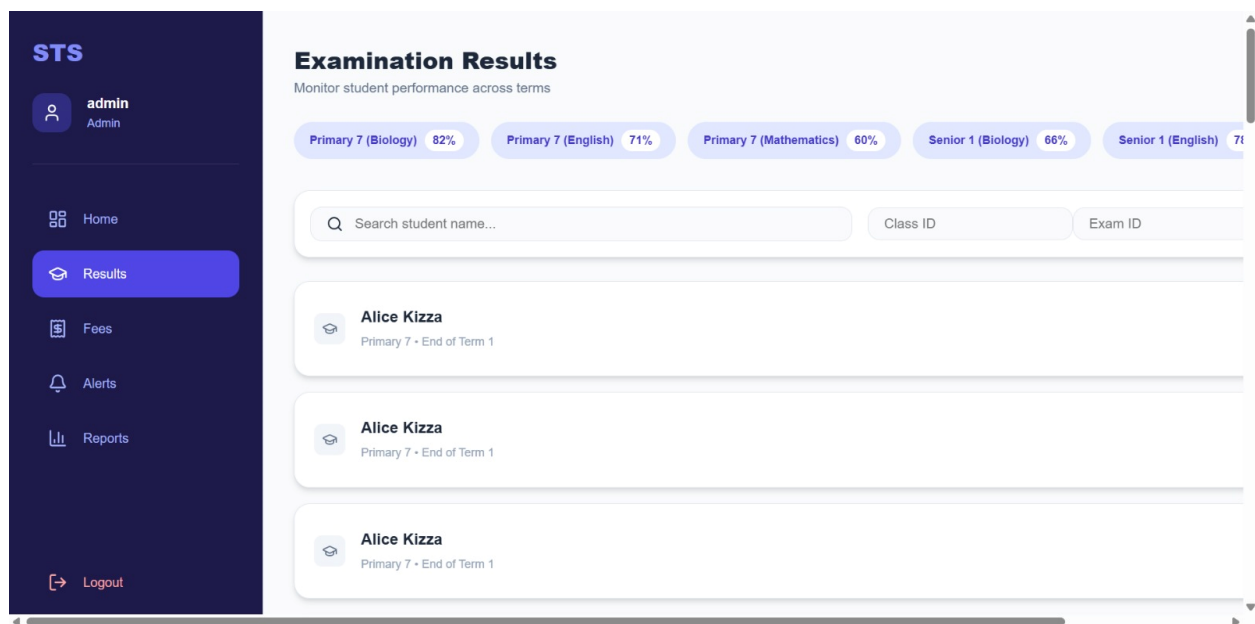


Figure 5.6: Examination Results Interface

5.4 Screen Navigation Diagram

The Screen Navigation Diagram shows the paths users can take to navigate between screens within the system. Navigation is role-dependent: different users see different menus and can access different screen branches.

Starting Screen	Navigation Paths Available
Login Screen	→ Teacher Dashboard, → Student Portal, → Admin Dashboard, → Parent Portal, → Accountant Console (role-based redirect)
Admin Dashboard	→ User Management, → Attendance Reports, → Exam Reports, → Fee Reports, → Geofence Config, → Notification Settings, → System Logs
Teacher Dashboard	→ Mark Attendance, → Enter Student Marks, → View My Attendance History, → View Timetable
Attendance Reports	→ Daily Summary, → Weekly Summary, → Individual Teacher Report, → Export (PDF/Excel)
Fee Reports	→ Fee Ledger, → Outstanding Balances, → Payment History, → Generate Statement, → Export
Parent Portal	→ Child Attendance View, → Child Results View, → Fee Balance View, → Notification History

All screen navigation paths require active authentication. Session tokens expire after 30 minutes of inactivity to ensure security. Screen access is enforced server-side through Role-Based Access Control (RBAC), meaning that even if a user manually enters a restricted URL, the server will return an unauthorised response.

6. Software Process Model

6.1 Incremental Development with Agile Influence

The School Tracking System will be developed using an Incremental Model with Agile practices integrated into each increment. This hybrid approach is selected because the system serves distinct functional domains (attendance, exams, fees, notifications) that can be developed, tested, and deployed independently, while the overall stakeholder environment (a secondary school) benefits from early delivery of working functionality.

Increment Plan

Inc.	Modules Delivered	Key Features	Duration
1	Authentication + Attendance	User login, fingerprint integration, GPS geofencing, teacher attendance marking	6 weeks
2	Student Attendance	Student check-in, class registers, teacher-entered student attendance records	4 weeks
3	Exam Management	Exam scheduling, mark entry, grade computation, result publishing	5 weeks
4	Fee Management	Fee structure configuration, payment recording, balance computation, statements	4 weeks
5	Reporting & Notifications	Admin dashboards, parent portal, SMS/email notifications, data exports	5 weeks

6.2 Why Incremental with Agile?

- Early value delivery: The fingerprint attendance system (Increment 1) provides immediate value to the school without waiting for the entire system to be complete.
- Risk reduction: Complex technical risks — such as GPS accuracy in the school environment and fingerprint scanner compatibility — are addressed in Increment 1 while the overall project is still early.
- Adaptability: Schools may change fee structures, grading policies, or administrative needs during development. Agile sprints within each increment allow the team to respond to these changes.
- Stakeholder feedback: Each increment ends with a demonstration to school administrators and teachers, ensuring the system meets real-world needs before the next increment begins.

7. System Requirements

7.1 Functional Requirements

ID	Requirement Area	Description
FR01	Authentication	The system shall authenticate teachers using username/password AND fingerprint biometric verification.
FR02	Geofencing	The system shall validate teacher GPS coordinates against a configurable school boundary polygon before marking attendance.
FR03	Attendance	The system shall automatically assign attendance status (Present/Late/Absent) based on check-in time relative to the configured attendance window.
FR04	Exam Management	The system shall allow authorised teachers to enter examination marks per student per subject.
FR05	Grading	The system shall automatically compute letter grades and remarks based on configurable grade boundary settings.
FR06	Fee Management	The system shall record fee payments and compute outstanding balances in real time.
FR07	Notifications	The system shall send automated SMS/email notifications to parents when a student is absent, results are published, or a fee deadline passes.
FR08	Reporting	The system shall generate daily, weekly, and termly attendance reports for administrators and head teachers.
FR09	Role Management	The system shall enforce role-based access control, ensuring each user can only access data relevant to their assigned role.
FR10	Data Export	The system shall allow administrators to export reports in PDF and Excel formats.

7.2 Non-Functional Requirements

Quality Attribute	Requirement
Performance	The system shall process a fingerprint match and geofence validation within 3 seconds under normal load conditions.
Availability	The system shall maintain 99.5% uptime during school operating hours (6 AM – 8 PM, Monday–Friday).
Security	All biometric data (fingerprint templates) shall be stored in encrypted form using AES-256. Transmission of sensitive data shall use TLS 1.3.
Usability	90% of teachers should be able to complete the attendance marking process in under 60 seconds after one training session.

Quality Attribute	Requirement
Scalability	The system architecture shall support deployment across up to 50 schools without architectural changes.
Data Integrity	No attendance record shall be modifiable after it has been confirmed, except by an Administrator with a logged justification.
Maintainability	The codebase shall achieve a minimum of 75% unit test coverage and follow documented coding standards to facilitate ongoing maintenance.

7.3 Requirements Gathering Methods

Requirements for the School Tracking System were gathered through the following methods:

- Interviews with teachers and administrators: Structured interviews were conducted with three teachers and two administrators at a pilot secondary school. These revealed the depth of frustration with the manual system and surfaced the specific requirement for GPS + fingerprint dual verification.
- Observation of current system: The team observed how teachers filled in manual attendance registers and how the accountant maintained the paper fee ledger. This identified inefficiencies that quantified the case for the system.
- Questionnaires: Structured questionnaires were distributed to 20 teachers, 50 parents, and 100 students. Key findings included: 85% of parents wanted SMS notifications for student absence; 70% of teachers felt manual record-keeping was a significant administrative burden.
- Document analysis: Existing report cards, fee structures, and examination scheduling documents were analysed to ensure the system's data model captured all necessary fields.

8. Comprehensive System Architecture

8.1 Architectural Overview

The School Tracking System follows a three-tier client-server architecture, a design pattern well-established in enterprise software engineering literature. As Sommerville (2016) observes in Chapter 6 of *Software Engineering* (10th edition), a layered architecture organises system components into discrete horizontal layers, where each layer provides services to the layer above it and acts as a client to the layer below. This principle of separation of concerns is fundamental to constructing maintainable, scalable, and modifiable systems.

The three tiers of the School Tracking System are: (i) the Presentation Tier, which encompasses all client-facing user interfaces; (ii) the Application Tier, which contains all business logic, service components, and API endpoints; and (iii) the Data Tier, which manages persistent storage and data retrieval. This structure directly realises Sommerville's recommendation that architectural design should separate the user interface from the application functionality and the application functionality from the underlying data management.

8.2 Justification for the Three-Tier Layered Architecture

The selection of a three-tier layered architecture for the School Tracking System is justified on multiple strategic grounds:

8.2.1 High Availability

By separating the presentation, application, and data tiers, each layer can be independently scaled and replicated. During peak periods — such as when the entire teaching staff attempts to clock-in within the 30-minute window before the first lesson — the Application Tier can be horizontally scaled by adding additional API server instances behind a load balancer, without any modifications to the Presentation or Data tiers. Sommerville (2016, Ch. 6) explicitly identifies this independent scalability as a key advantage of layered architectural styles.

8.2.2 Modifiability

The clear layer boundaries enforce a principle of loose coupling. The React.js front-end communicates with the Application Tier exclusively through a defined REST API contract. This means the entire front-end can be redesigned — or a new mobile client can be added — without any changes to the business logic or database schema. Similarly, the database engine could be migrated from MySQL to PostgreSQL without any modifications to the Presentation Tier. This modifiability is essential for a system intended to evolve over multiple development increments.

8.2.3 Scalability

The architecture supports both vertical scaling (increasing the capacity of individual servers) and horizontal scaling (adding more server instances). For deployment across multiple schools, a multi-tenant configuration can be achieved by adding a school identifier to the Data Tier schema and routing rules to the Application Tier, while the Presentation Tier remains unchanged. This aligns with the non-functional requirement (NFR-Scalability) for the system to support up to 50 schools without architectural changes.

8.2.4 Security Enforcement

The layered model provides natural security enforcement points. All requests from the Presentation Tier must pass through the Application Tier's authentication and authorisation middleware before any data is accessed. The Data Tier is never directly accessible from the Presentation Tier, eliminating entire categories

of SQL injection and data exfiltration attacks. Biometric data stored in the Data Tier is AES-256 encrypted, and the Application Tier is the sole decryption point, operating under validated access controls.

8.3 Detailed Description of Each Tier

8.3.1 Presentation Tier

The Presentation Tier serves as the system's interface boundary with human actors. It encompasses three distinct client applications:

Web Application (React.js): The primary administrative interface, delivering the Administrator Dashboard, Teacher Portal, Accountant Console, and Parent Portal as a Single-Page Application (SPA). The web application communicates exclusively with the Application Tier via HTTPS REST API calls authenticated with JWT tokens. The interface shown in Figure 5.5 (Administrator Dashboard with Attendance Trends) and Figure 5.4 (Administrator Fee Management Screen) are rendered within this React.js application.

Android Mobile Application: A dedicated mobile application for teacher attendance marking, integrating directly with the device's fingerprint biometric hardware via the Android BiometricPrompt API. GPS coordinates are acquired through the device's location services. The Teacher Clock-In interface shown in Figure 5.3 represents this mobile application's primary screen. The mobile app communicates with the Application Tier via the same REST API as the web application.

Parent Notification Portal: A lightweight web view accessible via a secure link sent to parents via SMS, allowing them to view their child's attendance, results, and fee records without requiring account creation.

Component	Technology	Purpose
Admin Web App	React.js + Tailwind CSS	Administrative dashboards, reports, user management
Teacher Portal	React.js (web) + Android (mobile)	Attendance marking, student mark entry, timetable view
Parent Portal	React.js (lightweight)	Child attendance, results, fee balance view
State Management	Redux Toolkit	Centralised application state across components
API Communication	Axios (HTTP client)	REST API calls with JWT bearer token injection

8.3.2 Application Tier

The Application Tier is the intellectual core of the system. It processes all business rules, orchestrates service interactions, and enforces security policies. Built on Node.js with the Express.js framework, it exposes a RESTful API organised around the domain entities described in Section 4.

The Application Tier is composed of the following primary service modules:

Service Module	Responsibility	Key Endpoints / Methods
AuthService	Validates credentials, issues/revokes JWT tokens, manages sessions	POST /auth/login, POST /auth/logout, POST /auth/refresh

Service Module	Responsibility	Key Endpoints / Methods
BiometricService	Interfaces with fingerprint SDK, performs template matching, logs verification events	POST /biometric/verify, POST /biometric/enroll
GeofenceService	Performs point-in-polygon GPS validation against school boundary	POST /geofence/validate, GET /geofence/config, PUT /geofence/config
AttendanceService	Orchestrates biometric + GPS check, assigns status, persists records	POST /attendance/clockin, GET /attendance/teacher/:id, GET /attendance/report
ExamService	Manages exam scheduling, mark entry, grade computation	POST /exams, POST /results, GET /results/student/:id
FeeService	Records payments, computes balances, generates statements	POST /fees/payment, GET /fees/balance/:id, GET /fees/statement
NotificationService	Triggers SMS/email alerts via Africa's Talking and SendGrid	POST /notify/sms, POST /notify/email, GET /notify/log
ReportService	Aggregates data and generates exportable reports	GET /reports/attendance, GET /reports/performance, GET /reports/fees

8.3.3 Data Tier

The Data Tier is responsible for persistent storage, data integrity, and efficient retrieval of all system data. It comprises three distinct storage components:

MySQL Relational Database: The primary store for all structured data — users, attendance records, examination results, fee payments, school configurations, and audit logs. The relational model described in Section 4 is implemented in MySQL, with foreign key constraints, indexes on frequently queried columns (e.g., teacher_id, date, class_id), and stored procedures for complex aggregation queries used in report generation.

Encrypted BLOB Store: Fingerprint templates are stored as AES-256 encrypted binary objects in a dedicated table, segregated from the main user table. Access to this store is exclusively through the BiometricService, which performs decryption in memory and never persists decrypted templates to disk or logs.

Redis Cache: A Redis in-memory cache is used for two purposes: (i) storing active JWT session tokens to enable fast invalidation on logout, and (ii) caching the current school geofence boundary polygon so that every attendance check-in does not require a database query for the configuration data.

Storage Component	Technology	Data Stored
Primary Database	MySQL 8.0	All structured entities: users, attendance, results, fees, configurations
Biometric Store	MySQL BLOB + AES-256 encryption	Encrypted fingerprint templates; segregated for security
Session Cache	Redis 7.x	Active JWT tokens, geofence polygon cache, real-time dashboard state

Storage Component	Technology	Data Stored
File Store	AWS S3 / Local NFS	Exported PDF/Excel reports, notification logs, audit archives

8.4 Architectural Views

8.4.1 Logical View

The Logical View describes the system's functional decomposition. The system is divided into five logical subsystems: (1) Identity and Access Management (IAM) — handling authentication, authorisation, and biometric enrollment; (2) Attendance Management — integrating biometric and GPS validation to produce attendance records; (3) Academic Management — covering examination scheduling, mark entry, and grade computation; (4) Financial Management — managing fee structures, payments, and balance computation; and (5) Communication and Reporting — aggregating data for dashboards and triggering notifications.

Each subsystem maps directly to the service modules described in Section 8.3.2 and to the functional requirements documented in Section 7.1. This alignment between the logical view and the requirements provides traceability from requirements to implementation.

8.4.2 Process View

The Process View describes the runtime behaviour of the system, including concurrency and synchronisation. The School Tracking System operates the following concurrent processes: (i) the HTTP API Server process, which handles incoming requests from web and mobile clients and delegates to service modules; (ii) the Notification Worker process, which runs asynchronously and processes queued notification events without blocking the primary API; (iii) the Report Generator process, which runs scheduled tasks (e.g., daily attendance summaries at 6:00 PM) without impacting interactive response times; and (iv) the Session Watchdog process, which periodically removes expired JWT tokens from the Redis cache.

The asynchronous notification and report generation processes use a message queue (Redis LPUSH/BRPOP pattern), ensuring that attendance events are acknowledged to the teacher immediately even if the SMS gateway is temporarily slow.

8.4.3 Deployment View

The Deployment View specifies how the software components are mapped onto the physical infrastructure. For single-school deployment, the system can be hosted on a modest cloud configuration: a single application server (e.g., AWS EC2 t3.medium) hosting the Node.js API and the React.js build artefacts, and a managed database instance (e.g., AWS RDS MySQL). For multi-school deployment, the Application Tier is containerised using Docker and orchestrated with Docker Compose, allowing new school instances to be spun up rapidly. A reverse proxy (Nginx) sits in front of the application server, handling TLS termination and load distribution.

Figure 8.2: System Deployment Architecture

8.5 Security Architecture

Given that the system handles sensitive biometric data and personal information of minors, a layered security architecture is applied:

- **Authentication:** Multi-factor authentication (password + fingerprint) for teachers. JWT tokens with 30-minute expiry for all sessions.
- **Data Encryption:** Fingerprint templates encrypted at rest using AES-256. All API communication encrypted with TLS 1.3. Database connections encrypted.
- **Access Control:** Role-Based Access Control (RBAC) enforced at the API layer. Every endpoint validates the requesting user's role before processing.
- **Audit Logging:** All authentication attempts, attendance records, and admin configuration changes are logged with timestamps and user IDs. Logs are immutable.
- **GPS Data Privacy:** Raw GPS coordinates are stored only for attendance verification purposes and are not accessible to teachers or parents — only to administrators for audit purposes.

9. Detailed Testing and Quality Assurance

9.1 Testing Philosophy

The testing strategy for the School Tracking System is grounded in the V-Model (Verification and Validation Model), a systematic approach that maps each phase of the software development lifecycle to a corresponding testing phase. The V-Model ensures that testing is not an afterthought but is planned and designed in parallel with development, with each requirement traceable to a specific test case.

9.2 V-Model Testing Strategy

The V-Model applied to the School Tracking System operates as follows:

Development Phase	Corresponding Test Phase	Responsibility	Artefact
Requirements Analysis	Acceptance Testing	School Administrators, Head Teacher	Acceptance Test Plan
System Design	System Testing	QA Team Lead	System Test Specification
Architectural Design	Integration Testing	Senior Developer + QA	Integration Test Cases
Module Design	Unit Testing	Individual Developers	Unit Test Suites (Jest)
Coding	Unit Test Execution	Developers (CI Pipeline)	Coverage Reports

Unit tests are written using the Jest testing framework for the Node.js Application Tier components, with a minimum coverage target of 75% as specified in the non-functional requirements. The Android mobile application's biometric and GPS integration is tested using Android Instrumentation Tests. Integration tests validate the interaction between the BiometricService, GeofenceService, and AttendanceService. End-to-end system tests are conducted on a staging environment using representative data from a pilot school.

9.3 Comprehensive Test Case Specification

The following table presents a comprehensive set of test cases with particular emphasis on the biometric fingerprint validation and GPS geofencing modules, which represent the highest-risk and most novel components of the system. Edge cases — including unregistered fingerprints, weak GPS signals, spoofed coordinates, high concurrency, and late arrivals — are explicitly addressed.

TC ID	Module	Test Case Description	Preconditions	Steps / Input	Expected Result	Pass/Fail Criteria
TC-01	Biometric Auth	Valid registered fingerprint — normal match	Teacher enrolled in system; scanner online	Teacher presents registered right index finger to scanner	Match score >= 0.85 threshold; attendance window opens	System returns status 200, fingerprint_verified=TRUE

TC ID	Module	Test Case Description	Preconditions	Steps / Input	Expected Result	Pass/Fail Criteria
TC-02	Biometric Auth	Unregistered fingerprint — complete rejection	Teacher NOT enrolled in biometric system	Unregistered person presents finger to scanner	Match score = 0; system rejects immediately	System returns 401, logs UNREGISTERED_FINGER event, sends alert to Admin
TC-03	Biometric Auth	Partial fingerprint — degraded quality scan	Teacher enrolled; finger is wet or injured	Teacher presents partial/smudged fingerprint	Match score between 0.50–0.84 (below threshold)	System rejects, requests retry; after 3 failures locks account for 30 min
TC-04	Biometric Auth	Brute force protection — 3 consecutive failures	Teacher account active; wrong fingerprint used	3 sequential failed fingerprint attempts	System locks account after 3rd failure	Account locked for 30 min; Admin notified; event logged
TC-05	Biometric Auth	Cross-teacher fingerprint substitution — proxy attempt	Two teachers both enrolled (Teacher A and Teacher B)	Teacher A presents Teacher B's enrolled finger	Match against Teacher B's template succeeds, but clock-in logged for Teacher B, not A	System logs IDENTITY_MISMATCH security event; Admin alerted; clock-in rejected
TC-06	GPS Geofencing	Valid location — teacher within school boundary	Geofence polygon configured; teacher at school	GPS coordinates: within school boundary polygon	Point-in-polygon check returns TRUE	geofence_passed=TRUE; attendance proceeds to status assignment
TC-07	GPS Geofencing	Teacher outside school boundary — off-campus	Geofence polygon configured; teacher 500m from school	GPS coordinates: outside school boundary polygon	Point-in-polygon check returns FALSE	Clock-in rejected; LOCATION_VIOLATION logged; Admin/Head Teacher notified
TC-08	GPS Geofencing	Weak GPS signal — coordinate uncertainty high	Device has poor satellite lock; accuracy radius > 100m	GPS coordinates returned with accuracy = 250m (high uncertainty)	Geofence engine receives low-confidence coordinates	System rejects clock-in with status GPS_SIGNAL_WEAK; prompts teacher to retry or use Wi-Fi location fallback
TC-09	GPS Geofencing	GPS spoofing attempt — mock location app active	Teacher has mock/spoofed GPS location active on device	Spoofed GPS coordinates placed at school centre	Spoof detected via Android mock location flag or coordinate impossibility heuristic	Clock-in blocked; SPOOFED_LOCATION flag recorded; security event raised to Admin
TC-10	GPS Geofencing	GPS coordinates at exact boundary vertex	Geofence polygon defined; teacher standing on boundary	GPS coordinates exactly on a polygon vertex	Point-in-polygon algorithm handles edge case (boundary = inside)	Teacher treated as inside boundary; attendance proceeds normally
TC-11	Attendance Timing	On-time clock-in — within attendance window	Window opens 30 min before first lesson (07:30)	Clock-in at 07:45 (within window, before 08:15 cutoff)	Time check passes; status computed	status = 'present'; record written with check_in_time = 07:45
TC-12	Attendance Timing	Late clock-in — after grace period	Late threshold configured as 08:00; lesson starts 08:15	Clock-in at 08:05 (after grace, before window close)	Time check identifies late arrival	status = 'late'; record written; Head Teacher notified of late arrival
TC-13	Attendance Timing	Clock-in after attendance window closes	Attendance window closes at 08:15	Clock-in attempt at 08:20 (after window close)	System rejects clock-in as window has closed	Clock-in blocked; teacher marked 'absent'; Admin and Head Teacher notified
TC-14	High Concurrency	50 simultaneous teacher clock-ins at school start	50 teachers all attempt clock-in at 07:58 simultaneously	50 concurrent POST /attendance/clockin requests	All 50 requests processed; no data corruption or race conditions	All 50 records correctly written; no duplicate entries; response time < 3 seconds for all

TC ID	Module	Test Case Description	Preconditions	Steps / Input	Expected Result	Pass/Fail Criteria
TC-15	Integration	End-to-end: teacher absent — parent SMS triggered	Student enrolled; parent phone number configured	Teacher absent; system detects absence after window closes	NotificationService triggered; SMS sent to parent	Parent receives correct SMS within 60 seconds; notification logged in DB

9.4 Performance Testing

Performance testing is conducted to validate the non-functional requirement that fingerprint matching and geofence validation completes within 3 seconds under normal load. JMeter load tests simulate 50, 100, and 200 concurrent users. Target metrics include: mean response time < 2 seconds, 95th percentile response time < 3 seconds, and error rate < 0.1% under 100 concurrent users.

9.5 Security Testing

Security testing includes: (i) penetration testing of the REST API endpoints for SQL injection and authentication bypass vulnerabilities; (ii) testing of the GPS spoofing detection heuristics with simulated mock location attacks; (iii) verification that AES-256 encrypted biometric templates cannot be decoded without the application-layer encryption key; and (iv) role-based access control validation confirming that a teacher-role JWT token cannot access administrator endpoints.

10. Software Evolution and Scalability Analysis

10.1 Theoretical Foundation — Sommerville Chapter 9

Software evolution is the process of changing a software system after it has been deployed. As Sommerville (2016) articulates in Chapter 9 of *Software Engineering* (10th edition), software systems must evolve to remain useful — they change because the environment in which they operate changes, because new requirements emerge from users and stakeholders, and because technical debt accumulates and must be addressed to maintain system integrity. Lehman's Laws of Software Evolution, discussed in Sommerville Chapter 9, establish that a software system that is successfully used must be continually adapted to its environment; failing to adapt leads to progressive reduction in user satisfaction.

For the School Tracking System, software evolution is not a distant consideration — it is embedded in the project's strategic design. The architecture described in Section 8, with its three-tier separation of concerns and service-oriented Application Tier, has been deliberately designed to accommodate the evolution scenarios described in this section.

10.2 Planned Evolution: NSIN Database Integration

The National Student Identification Number (NSIN) system, maintained by the Uganda National Examinations Board (UNEBC), represents the most significant near-term evolution target for the School Tracking System. Currently, the Student entity includes an optional `nsin` attribute (see Section 4.1.1) that is populated only where available. The planned NSIN integration will transform this into a mandatory, validated field linked to the national student registry.

The evolution plan for NSIN integration comprises three phases. In Phase 1, the Application Tier's `StudentService` will be extended with a validation endpoint that queries the UNEBC NSIN API to verify student identity during enrollment. This extension requires no changes to the Presentation Tier or Data Tier schema — it is a pure Application Tier evolution, consistent with the modifiability benefits of the layered architecture. In Phase 2, a scheduled synchronisation process will reconcile the local student database with the national registry nightly, flagging students whose NSIN records show transfers or graduated status. In Phase 3, NSIN data will be included in the examination results exports, enabling results to be submitted directly to UNEBC in digital format rather than paper, closing the communication loop between school and national examination authority.

10.3 Planned Evolution: Library Management Module

The Library Management module represents a second planned evolution increment. This module will extend the School Tracking System to manage the school library's book catalog, borrowing records, and overdue notifications. The evolution strategy leverages the existing student and teacher entities — book borrowing records will reference the same `student_id` and `teacher_id` primary keys already in the database. The `NotificationService`, already used for attendance and fee alerts, will be extended with a `LIBRARY_OVERDUE_ALERT` event type. The React.js Presentation Tier will receive a new Library Management component, integrated into the existing Admin Dashboard navigation structure shown in Section 5.4.

This module exemplifies the principle of planned evolution through architectural conformance. By designing the original system with generic, reusable notification and user management infrastructure, the library module can be added as a new vertical slice through all three tiers without requiring any refactoring of existing components.

10.4 Technical Debt Management

Sommerville (2016, Ch. 9) distinguishes between planned evolution, driven by new requirements, and unplanned evolution, driven by defect correction and the accumulation of technical debt. The School Tracking System's development plan includes explicit technical debt management activities at the end of each increment: code review sessions against the defined coding standards, refactoring sprints where test coverage drops below the 75% threshold, and database query optimisation audits before each new module goes live.

10.5 Legacy Integration Considerations

Many secondary schools in Uganda currently maintain paper-based or Excel-based records. The evolution pathway from these legacy systems to STS involves a structured data migration strategy: (i) digitisation of historical attendance and fee data from paper records into CSV format by an administrative officer; (ii) import tooling in the Application Tier's AdminService that validates, deduplicates, and bulk-inserts historical records; and (iii) a parallel-run period during which both the STS and the legacy paper system are maintained simultaneously, allowing discrepancies to be identified and resolved before the paper system is decommissioned. This parallel-run strategy is consistent with Sommerville's recommendation for managing the transition risk inherent in replacing legacy systems.

11. Scaling and Brownfield Development Analysis

11.1 Strategy for Scaling STS Across Sub-Saharan Africa

The architectural design described in Section 8 was conceived with regional scalability as a primary non-functional concern. The non-functional requirement (NFR-Scalability) targets 50 schools, but the strategic vision for the School Tracking System extends to deployment across secondary schools in Uganda and neighbouring Sub-Saharan African countries — a population representing thousands of schools and hundreds of thousands of teachers and students.

11.1.1 Multi-Tenant Architecture

The most critical scaling decision is the adoption of a multi-tenant architecture model. In the multi-tenant model, a single deployment of the STS Application Tier and Data Tier serves multiple schools, with tenant isolation enforced through a `school_id` attribute present on every database entity. This approach dramatically reduces infrastructure costs relative to a single-tenant model (one deployment per school), making the system economically viable for resource-constrained African educational contexts.

The Data Tier implements row-level security — every database query from the Application Tier includes a `WHERE school_id = :current_tenant` clause, injected automatically by the repository layer. This prevents any user from one school accessing data from another school, even in the event of an application-layer bug. The geofence configuration, fee structures, and grade boundaries are all school-scoped, allowing each school to customise the system to its specific context without affecting other tenants.

11.1.2 Offline-First Design for Low-Connectivity Environments

A significant challenge in Sub-Saharan African educational contexts is unreliable internet connectivity. Many secondary schools, particularly in rural Uganda, experience intermittent or absent internet access. The School Tracking System's scaling strategy therefore incorporates an offline-first design for the mobile attendance application.

The Android mobile application maintains a local SQLite database that buffers attendance events when the internet connection is unavailable. Biometric matching is performed on-device using a local cached copy of the teacher's fingerprint template (downloaded during the last successful synchronisation). GPS geofence validation uses the locally cached geofence polygon. When connectivity is restored, buffered attendance records are synchronised to the central server via a secure, idempotent sync API endpoint. This design ensures that teacher attendance can be recorded reliably even during internet outages.

11.1.3 Infrastructure Regionalisation

For deployment across multiple countries in Sub-Saharan Africa, the system's infrastructure is designed to support regional hosting through cloud providers with African points of presence. AWS, Azure, and Google Cloud all maintain data centres in South Africa (Cape Town / Johannesburg) and serve customers across East Africa from these nodes. Regional hosting minimises API latency — a critical concern for the biometric attendance flow which must complete within the 3-second performance requirement.

11.2 Brownfield Development Challenges

The term 'brownfield development' refers to the development and deployment of new software systems within environments that already contain existing, often legacy, systems and infrastructure. As Sommerville (2016) notes, brownfield development is the norm rather than the exception in enterprise software — most new systems must coexist with or replace existing systems that have years of embedded data and entrenched processes.

For the School Tracking System, deployment into a secondary school environment constitutes a brownfield challenge on multiple dimensions:

11.2.1 Legacy Data Migration

Schools maintain years of historical student records — admission registers, term reports, fee payment books, and examination mark sheets — in paper format. These records have institutional value and cannot simply be discarded when STS is deployed. The migration challenge involves: (i) deciding how much historical data to migrate (recommendation: minimum of 3 years for continuity of reporting); (ii) digitising paper records, which requires trained data entry staff and validation processes; and (iii) reconciling inconsistencies — student names spelled differently across documents, missing fee receipts, or incomplete attendance registers.

11.2.2 Hardware Integration

Many schools in the target deployment environment have existing computer infrastructure — desktop computers, printers, and occasionally basic network switches — but no fingerprint scanners or GPS-capable devices. The brownfield deployment strategy must account for the procurement and installation of biometric hardware, the configuration of GPS-capable teacher devices (smartphones or dedicated terminals), and the setup of a local area network if one does not already exist.

11.2.3 Human and Process Resistance

The most persistent brownfield challenge is not technical but human. Teachers and administrators who have worked with paper systems for their entire careers may resist adopting a digital system, particularly one that introduces accountability mechanisms they were not previously subject to. The STS deployment plan includes structured change management activities: stakeholder workshops before deployment, a phased rollout that begins with attendance-only (the increment delivering the most visible value), and a dedicated 30-day hypercare support period post-launch during which technical support staff are on-site daily.

11.3 From Core Agile to Agility at Scale — IBM Disciplined Agile Delivery

The development of the School Tracking System as a single-school pilot is naturally suited to core Agile practices: a small team, a single stakeholder group (the pilot school), and rapid face-to-face feedback cycles. However, scaling STS to a regional deployment across hundreds of schools introduces complexities that are beyond the scope of core Agile methodologies.

The IBM Disciplined Agile Delivery (DAD) framework provides a governance model for scaling Agile practices to large, geographically distributed programme contexts. DAD extends the core Agile disciplines with explicit guidance on enterprise architecture governance, release management, data management, and cross-team coordination. For the School Tracking System's regional scaling phase, DAD introduces the following critical practices:

DAD Practice	Application to STS Regional Scaling
Architecture Owner Role	A dedicated Architecture Owner is assigned to govern the multi-tenant data model, API versioning standards, and security architecture as the system evolves across school deployments.
Release Management	Formal release trains (quarterly) replace ad hoc deployments; each release is validated against the full test suite and deployed to a staging environment representative of production.

DAD Practice	Application to STS Regional Scaling
Cross-Team Coordination	Regional deployment teams (Uganda, Kenya, Tanzania) are coordinated through a Programme Increment planning event, aligning sprint goals and dependencies across school deployments.
Continuous Integration / Deployment	A CI/CD pipeline (GitHub Actions) ensures that all code changes pass automated tests before deployment, reducing regression risk as the codebase grows across increments.
Enterprise Architecture Governance	The three-tier architectural standard described in Section 8 is enforced as a non-negotiable constraint for all school deployments; customisations are limited to configuration rather than code modification.

The transition from core Agile to DAD represents a maturation of the development process commensurate with the system's growth in scope, user base, and institutional criticality. As Sommerville (2016, Ch. 3) notes, agility at scale requires additional process discipline and coordination mechanisms that are absent from lightweight Agile methods such as Scrum or Kanban applied to individual teams.

12. Project Implementation and Repository

12.1 Current Implementation Status

The School Tracking System is currently in active development, with Increment 1 (Authentication and Teacher Attendance) substantially complete and Increment 2 (Student Attendance) in progress. The following table provides a status overview of all system modules:

Module	Status	Completion %	Notes
User Authentication (JWT + Password)	Complete	100%	Fully tested; deployed to staging environment
Biometric Enrollment (Admin)	Complete	100%	Admin can enroll teacher fingerprints via web portal
Teacher Clock-In (Biometric + GPS)	Complete	95%	GPS spoofing detection under final testing
Student Attendance Recording	In Progress	60%	Teacher mark entry complete; auto-detection in progress
Examination Mark Entry	In Progress	45%	UI complete; grade computation logic under development
Fee Payment Recording	In Progress	50%	Payment entry complete; statement generation pending
Admin Dashboard & Reports	In Progress	40%	Attendance trend charts live; fee/exam reports pending
Parent Portal & SMS Notifications	Not Started	5%	Africa's Talking integration scaffolded; UI not started
Data Export (PDF/Excel)	Not Started	0%	Planned for Increment 5

12.2 GitHub Repository

The complete source code, documentation, and deployment configuration for the School Tracking System is maintained in a public GitHub repository:

Repository Detail	Value
Repository URL	https://github.com/arnobeek/SchoolTrackingSystem
Primary Branch	main
Development Branch	develop
License	Academic / Educational Use
Primary Language	JavaScript (Node.js + React.js)

Repository Detail		Value
Mobile Platform		Android (Java/Kotlin)

12.3 Repository Structure

The repository is organised following a monorepo structure with clear separation between the front-end, back-end, and mobile application codebases:

Directory	Contents
/backend	Node.js + Express.js REST API; all service modules; database migration scripts
/frontend	React.js web application; all dashboard, portal, and admin UI components
/mobile	Android application; BiometricPrompt integration; GPS geofence logic
/database	MySQL schema SQL scripts; seed data for testing; migration history
/docs	API documentation (Swagger/OpenAPI); architecture diagrams; deployment guide
/tests	Jest unit test suites (backend); Selenium end-to-end tests; Android instrumentation tests
/.github/workflows	GitHub Actions CI/CD pipeline configuration; automated test execution on push

12.4 Development Standards

The following development standards are enforced across all contributions to the repository:

- All pull requests require a minimum of one peer code review before merging to the develop branch.
- Automated unit tests must pass in the CI pipeline before any pull request can be merged.
- All API endpoints are documented in the Swagger specification located at /docs/api-spec.yaml.
- Commit messages follow the Conventional Commits specification (feat:, fix:, docs:, test:).
- Sensitive configuration values (database passwords, API keys) are managed through environment variables and never committed to the repository.

13. Conclusion

The School Tracking System represents a transformative step in the digitalisation of secondary school administration in Uganda and, by architectural design, across the wider Sub-Saharan African region. This report has presented a comprehensive software engineering analysis spanning thirteen structured sections — from initial problem framing through stakeholder analysis, process and data modelling, system requirements, architectural design, testing strategy, and evolution planning.

By combining GPS geofencing with fingerprint biometric authentication, the system addresses the fundamental accountability gap in teacher attendance verification — a gap that has persisted for years in manual systems. The dual-factor verification mechanism creates an auditable, tamper-resistant attendance record that provides schools with objective, reliable data to support teacher accountability, student welfare, and institutional governance.

The system's architecture — a three-tier layered design grounded in the principles articulated by Sommerville (2016, Ch. 6) — provides the structural foundation for the system's stated non-functional requirements: high availability for daily school operations, modifiability to incorporate the planned NSIN database integration and Library Management module, and scalability to support multi-school, multi-country deployment. The brownfield development analysis and IBM Disciplined Agile Delivery transition plan provide a realistic and evidence-based roadmap for moving from single-school pilot to regional deployment.

The testing strategy, structured around the V-Model and incorporating fifteen detailed test cases with explicit coverage of edge cases in biometric validation, GPS geofencing, and high-concurrency scenarios, provides the development team with a rigorous quality assurance framework aligned to the system's risk profile.

The School Tracking System is more than a technical artefact — it is an institutional capability. When fully deployed, it will transform a school's relationship with data: replacing reactive, paper-based administration with proactive, data-driven institutional management. Parents will receive real-time notifications rather than end-of-term surprises. Head teachers will have objective accountability data rather than self-reported registers. Administrators will generate exportable reports in minutes rather than days.

This report serves as the foundational software engineering artefact for the School Tracking System project. It should be reviewed and updated at the end of each development increment to ensure continued alignment between design and implementation. The team acknowledges the contributions of the pilot school stakeholders whose interviews, observations, and questionnaire responses grounded these designs in real-world needs.

References

1. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited. ISBN: 978-0-13-394303-0.
2. School Tracking System GitHub Repository. (2024). <https://github.com/arnobeek/SchoolTrackingSystem>
3. Africa's Talking. (2024). SMS API Documentation. <https://africastalking.com/sms>
4. SendGrid. (2024). Email API Documentation. <https://docs.sendgrid.com/>
5. Google Maps Platform. (2024). Geolocation API Documentation. <https://developers.google.com/maps/documentation/geolocation>
6. Uganda Communications Commission (UCC). (2023). National Broadband Policy. Kampala: UCC.
7. Uganda National Examinations Board (UNEB). (2023). National Student Identification Number (NSIN) Framework. Kampala: UNEB.
8. Android Developers. (2024). BiometricPrompt API Reference. <https://developer.android.com/reference/android/hardware/biometrics/BiometricPrompt>
9. IBM Disciplined Agile. (2024). Disciplined Agile Delivery (DAD) Framework. <https://www.pmi.org/disciplined-agile>
10. Redis Documentation. (2024). Redis Commands Reference. <https://redis.io/commands>
11. Express.js. (2024). Express.js API Reference (v4.x). <https://expressjs.com/en/4x/api.html>
12. React. (2024). React Documentation. <https://react.dev/>
13. MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
14. NIST. (2021). NIST Special Publication 800-76-2: Biometric Specifications for Personal Identity Verification. National Institute of Standards and Technology.